

AEMaaCS Engineering Best Practices

Developers & Technical Architects – SHRSS Volume

1. Role & Responsibilities

As an AEMaaCS engineer or technical architect on SHRSS you:

- Design and implement **components, services, integrations, and APIs** aligned with AEMaaCS cloud patterns.
- Own the **technical quality** of Jobs/Events/News/Locations and career experiences.
- Guide indexing, caching, and dispatcher/CDN behavior to meet performance and SLAs.
- Partner with authors, DAM, and DevOps to ensure features are **operable, secure, and maintainable**.

For shared context across roles, use the **Consolidated All-Roles Volume**.

2. Mental Models

2.1 AEMaaCS as a “Managed Runtime”

- Adobe manages:
 - AEM binaries, OSGi framework, datastore, Mongo-based distribution layer, container stack.
 - Horizontal scaling, zero-downtime updates, and security patching.
- You own:
 - Custom bundles, content models, Sling Models, and dispatcher rules.
 - Index definitions, integration code, and pipeline-safe packaging.

2.2 SHRSS Domain Model as CF-first

- Business concepts (Jobs, Events, News, Locations) are **Content Fragments**, not pages.
- Components and pages:
 - Orchestrate and display CFs.
 - Provide navigation, search, filtering, and UX logic.
- Integrations (Workday, DPLT) treat CFs as **view models** over source-of-truth systems.

3. Project Structure & Packaging

3.1 Modules and Responsibilities

- `core`
 - Sling Models, OSGi services, servlets, schedulers.
 - Integration clients (Workday, TransPerfect, DPLT, Maps, etc.).
 - Domain logic for Jobs/Events/News/Locations.
- `ui.apps`

- Components, dialogs, policies, clientlibs.
 - Design configs under /conf/shrss.
- `ui.frontend`
 - Theme & site styling, front-end assets, JS frameworks if any.
- `ui.config`
 - OSGi configs & repo init for users/groups, service users, access control.
- `dispatcher`
 - Apache vhosts, dispatcher filters, cache rules, renders, and rewrites.
- `it.tests / ui.tests`
 - Integration tests using AEM test client.
 - Cypress/Playwright UI tests for Cloud Manager UI testing.

Engineering guideline

Every change must live in one of these modules and move through pipelines; there are no “one-off” runtime changes.

4. Components & Sling Models – Practices & Patterns

4.1 Component Patterns

- Prefer **Core Components** as base:
 - Use experience fragments, Teaser, List, Carousel, Title, Text, etc., extended as needed.
- When creating new components:
 - Design **dialog fields** to be minimal and non-ambiguous.
 - Use **policies** for styling & behavior toggles; avoid exposing too many toggles directly to authors.

Anti-patterns

- Logic-heavy HTL templates.
- Components that directly write to the repository.
- Components that depend on specific node names or structure instead of resource types & contracts.

Reference

- [Components and the Page Editor](#)

4.2 Sling Model Practices

- Define clear **interfaces / public methods** for models used across components.
- Use constructor injection or `@OSGiService` sparingly; prefer field injection with explicit defaults.
- For CF-driven components:

- Accept a **CF reference or path** in the dialog.
- Resolve the CF once, then map fields to a DTO; do not re-resolve in each getter.

5. Integrations

5.1 Workday → Jobs

- Encapsulate Workday access behind a **service interface**:
 - WorkdayJobsService with methods like syncJobs(), getJobById().
- Use **service users** for repository writes; never admin sessions.
- Respect source of truth:
 - Workday is authoritative for job metadata.
 - AEM holds **decorations**: images, hot flags, SEO overrides.

SHRSS Overlay

- Jobs CFs under .../Content/Fragments/Jobs contain:
 - **System fields** (Job ID, Workday URL, location fields) – overwritten on sync.
 - **Author-only fields** (image, hot flag, SEO override, “Is API Data”) – never overwritten.
- Integration code must:
 - Use “Is API Data” to distinguish API-managed from manual jobs.
 - Never modify author-only fields during sync.

5.2 DPLT → Locations

- Treat DPLT as source of truth for location master data.
- Location CFs:
 - Must be updated only via agreed DPLT→AEM process or controlled tools.
- When building components:
 - Reference **Location CFs**, not free-text addresses.

5.3 Other Integrations (TransPerfect, Maps, etc.)

- Use **configurable endpoints** and externalized credentials.
- Log:
 - Request IDs, important parameters, and clear error messages.
 - Never log secrets or PII.

6. Indexing & Query Design

6.1 General Checklist

Before writing a query:

1. Define **use case** and **expected volume** (e.g., Jobs listing by location, up to 10k jobs globally).
2. Decide **filters & sort**:
 - Which fields are exact matches?
 - Which fields are ranges (dates)?
 - Where is fulltext needed?
3. Confirm **path constraints** (e.g., Jobs CFs under /content/dam/shrss/.../jobs).
4. Check **existing indexes** (e.g., damAssetLucene-*):
 - If needed, customize rather than creating new fulltext indexes.

References

- [Content Search and Indexing](#)
- [Search and indexing in AEM as a Cloud Service](#)

6.2 SHRSS Overlay – Jobs Example

For Jobs:

- CFs are dam:Asset with:
 - jcr:content/data/cq:model = Job model.
 - jcr:content/contentFragment = true.
- Common filters:
 - department, jobType, location, tags.
- Common sort:
 - postDate DESC, expirationDate filters.

Index strategy:

- Customize damAssetLucene-* (e.g., damAssetLucene-8-custom-1) with:
 - Property indexes for department, jobType, tags, postDate, expirationDate, cq:model.
 - ordered=true for postDate and expirationDate.
 - Fulltext on jobTitle, jobDescription where required.

For a full worked example, see the **SHRSS Indexing & Performance Volume**.

7. Performance & Caching

7.1 Dispatcher & CDN Alignment

- Design responses to be **cacheable by default**:
 - Avoid session/stateful logic for public pages.
 - Use query parameters or path segments that map to intuitive cache keys.

- Use **Surrogate-Control / Cache-Control** headers to control CDN behavior.

References

- [Caching in AEM as a Cloud Service](#)

7.2 SHRSS Overlay – Jobs/Events/News

- Jobs, Events, News lists are:
 - Good candidates for aggressive caching with invalidation on CF changes.
- When changing list/detail components:
 - Avoid introducing user-specific behavior (e.g., “saved jobs”) on heavily cached routes.
 - If needed, split personalization into separate endpoints or client-side enhancements.

8. Tooling & Local Development

- Use **AEM SDK** + local Dispatcher for development.
- Favor **remote debugging** of local SDK instances during complex issues.
- Use **RDE** for near-real-time validation on Cloud Service, but keep full integration and pipeline testing for non-prod environments.

References

- [Local Development Environment Set up](#)
- [Rapid Development Environments](#)

9. Engineering SHRSS Overlay Summary

- Design around **CFs as canonical data** for Jobs/Events/News/Locations.
- Treat external systems (Workday, DPLT) as **authoritative sources**; AEM is a consumer/enricher.
- Use **indexing and dispatcher** as first-class design concerns for high-traffic lists.
- Keep **components small, composable, and Core-Component-inspired**; SHRSS complexity belongs in integration layers and CF models, not HTL.

Use this volume alongside:

- The **Consolidated All-Roles Volume** (for cross-role alignment).
- The **Indexing & Performance Volume** (for query/index deep dives).
- The **SHRSS SDD** (for solution-level architecture and decisions).